

AGENT-TASKS: Content-Addressed Verification for Autonomous Coding Agents on Kubernetes

Author One*

Author Two

September 1, 2026

Abstract

[TODO: Write abstract last, after Sections 6, 9, and 13 are stable.]

1 Introduction

1.1 Problem

1.2 Why existing orchestration does not fit

1.3 Contributions

[TODO: Draft after Section 2 and Section 6 settle.]

*Primitive. Correspondence: `author@example.com`

2 The Specification-Only Organization

- 2.1 Verification capacity as the bound
- 2.2 Two checks, neither substitutable
- 2.3 Where the work goes
- 2.4 Verification compounds; review does not
- 2.5 The autonomy dial
- 2.6 Trust is purchased with compute

3 Motivation

- 3.1 Anatomy of the convergence loop
- 3.2 Why one-shot execution fails
- 3.3 Why agent frameworks fail
- 3.4 Requirements

4 Related Work

- 4.1 Workflow and pipeline engines
- 4.2 Build systems: our actual lineage
- 4.3 Agent frameworks and benchmarks
- 4.4 Isolation
- 4.5 Kubernetes extension patterns

5 Design Goals and Non-Goals

- 5.1 Goals
- 5.2 Non-goals

6 The Verification Ledger

- 6.1 Definitions
- 6.2 Path-scoped invalidation
- 6.3 fold and decide as pure functions
- 6.4 Termination
- 6.5 Comparison to one-shot execution

7 Declarative Workflow Specification

- 7.1 Capability versus composition
- 7.2 Declaring dependencies and invalidation
- 7.3 Resolution and version pinning
- 7.4 Admission without webhooks
- 7.5 Non-composable intermediate environments

9 Threat Model

- 9.1 Trust boundaries
- 9.2 Gate-gaming
- 9.3 Defaults that fail closed
- 9.4 Best-of-N as adversarial selection pressure
- 9.5 Confused deputy
- 9.6 Capability scoping
- 9.7 Residual risks

10 Multi-Candidate Execution

- 10.1 Diversity and failure independence
- 10.2 Selection
- 10.3 Why hunk-level composition fails
- 10.4 Fan out cheap, converge before expensive

11 Issue Tracker Integration

- 11.1 A provider-neutral status vocabulary
- 11.2 One-directional at both edges
- 11.3 Webhook plus reconciling sweep
- 11.4 Interactive mode

12 The Post-Merge Validation Loop

- 12.1 Intent validation is above the gaming layer
- 12.2 Failure is a normal outcome
- 12.3 Compounding trust
- 12.4 Sampled audit

13 Preliminary Evaluation

- 13.1 Setup
- 13.2 Convergence behaviour
- 13.3 Invalidation efficiency
- 13.4 Cost
- 13.5 Safety
- 13.6 Trust over time
- 13.7 Threats to validity

14 Limitations and Future Work

14.1 Limitations

Availability

An open-source implementation of AGENT-TASKS is available at <https://github.com/primitivecorp/agent-tasks> under the Apache-2.0 license. The repository’s `implementation.md` is the normative design specification the implementation is built against; this paper is the argument for it. Reference manifests reproducing the configurations described in Section 7 are included in Appendix A.

References

A Reference Manifests

[TODO: Populate from repository testdata once Phase 1 is green.]

B Invalidation Semantics

Definition 1 (Snapshot). A snapshot h is the content identity of the workspace: the git tree hash, with submodule and LFS pointers taken as tree content. Snapshots are immutable. An action either produces a new snapshot or leaves the current one unchanged.

Definition 2 (Gate key). For a gate g with resolved specification s_g and a snapshot h , the gate key is $\kappa(g, h) = H(h, s_g)$ for a collision-resistant hash H . A diff-scoped gate additionally includes the base revision b : $\kappa(g, h) = H(b, h, s_g)$.

Definition 3 (Ledger). The ledger is a partial function $\mathcal{L} : (g, \kappa) \rightarrow \{\text{PASS}, \text{FAIL}(e)\}$, where e is evidence. Absence is UNKNOWN, and UNKNOWN is what schedules a gate.

Definition 4 (Invalidation). Let an action transform h into h' with changed path set P , and let $I(g)$ be the glob set declared by gate g , which defaults to $\{\text{**}\}$. The result of g carries forward to h' if and only if $P \cap \text{paths}(I(g)) = \emptyset$; otherwise it is absent at h' . If P is reported truncated, P is taken to be every path. The rule applies to FAIL results exactly as to PASS.

Definition 5 (Fixed point). Let a task have driver d , blocking gates B and non-blocking gates N . The task is at its fixed point at snapshot h if and only if d has completed at least once, every $g \in B$ has $\mathcal{L}(g, \kappa(g, h)) = \text{PASS}$, and every $g \in N$ either has a result at h or has exhausted its infrastructure retries.

Definition 6 (Oscillation). Let $h_0, \dots, h_k$ be the lineage of distinct snapshots a task has occupied, with h_0 the base. An action producing $h' \neq h_k$ oscillates if and only if $h' = h_i$ for some $i < k$. An action producing $h' = h_k$ is a no-op and is not oscillation.

Proposition 1 (Bounded termination). *Let R be the cap on driver runs, F the number of gates that name a fix action, A the per-gate cap on fix attempts, G the number of gates, and E the cap on consecutive infrastructure failures of one step. Every task reaches its fixed point or escalates after at most*

$$((R + FA) + G(R + FA + 1))(E + 1) + c$$

executions, for a small constant c .

Proof sketch. Each decision either terminates (DONE, ESCALATE) or schedules one execution that strictly consumes a bounded quantity. Driver completions are bounded by R and fix completions by FA , so completed actions number at most $R + FA$. Each new snapshot requires a completed

action, so the lineage has at most $R + FA + 1$ entries. A gate re-runs only after a new snapshot has cleared its result, so gate completions are bounded by $G(R + FA + 1)$. An infrastructure failure increments a per-step counter that only a success of that step resets, so each step suffers at most E failures per success and E more before escalating, which gives the factor $E + 1$. Oscillation, the wall-clock budget and the token budget only tighten the bound. The implementation's property suite asserts this arithmetic directly against generated execution sequences. $\square$

[TODO: Tighten the constant c once the property tests pin it.]